

# Reviewer Pack — Minimal Rank of Categorified Symplectic Forms over DPLL Sear...

Entry 15f19a3d4b30 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 12:53:20 UTC

## Minimal Rank of Categorified Symplectic Forms over DPLL Search Tree Width

**Entry ID:** 15f19a3d4b30 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 12:53:20 UTC

### 1. Conjecture

**Field A** (mathematical branch): Categorification Theory (Symplectic Geometry) **Field B** (complexity object): Complexity Theory: DPLL Search Tree Width

#### Statement:

{‘sentence\_1’: ‘For every Boolean formula  $F$  with  $n$  variables, let  $S(F)$  be the set of its clauses.’, ‘sentence\_2’: ‘The categorified symplectic form rank of  $F$  is defined as the minimal number of generators for the free symplectic Lie algebra over  $S(F)$ .’, ‘sentence\_3’: ‘There exists a polynomial-time computable function  $f(n)$ , such that for all Boolean formulas  $F$  with  $n$  variables, the categorified symplectic form rank of  $F$  satisfies:  $\text{categorified\_symplectic\_rank}(F) = \Theta(f(n))$ .’}

#### Rationale (proposer’s reasoning):

{‘sentence\_1’: ‘Categorification theory extends algebraic structures to a categorical level, potentially revealing new connections between mathematical objects and complexity measures.’, ‘sentence\_2’: ‘Symplectic geometry has been used in geometric complexity theory to relate geometric properties of spaces to computational problems.’, ‘sentence\_3’: ‘The DPLL search tree width is a complexity measure for Boolean formulas that captures the size of the proof search space. The conjecture suggests that categorified symplectic forms might provide insights into this complexity measure.’}

**Taxonomy category:** Categorification\_Symplectic\_Geometry (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** a932c59c1a8591fe

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

The categorified symplectic form rank of Boolean formulas with  $n$  variables is  $\Theta(f(n))$  if the ratio of the actual rank to the expected rank from  $f(n)$  is within  $[0.7, 1.3]$  for at least 80% of all seeds.

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict   | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|-----------|------------|------------------|------------------|
| RELATIVIZATION | UNCERTAIN | 1.00       | HITS             | UNCERTAIN        |
| NATURAL_PROOFS | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |

### 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

### 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=-9, elapsed=208.0s

#### 5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_formula(n):
        clauses = []
        for _ in range(2**n):
            clause = [random.choice([f'x{i+1}', f'~x{i+1}']) for i in range(n)]
            clauses.append(clause)
        return clauses

    def symplectic_form_rank(clauses):
        # Simplified version of computing the rank
        return len(set(tuple(sorted(c)) for c in clauses))

    n = random.choice([5, 10, 15, 20, 30, 40])
    formula = generate_formula(n)
    rank = symplectic_form_rank(formula)

    # Placeholder function for f(n) - replace with actual computation
    def f(n):
        return n**2

    expected_rank = f(n)
    ratio = rank / expected_rank if expected_rank != 0 else float('inf')

    metric_name = "categorified_symplectic_form_rank"
    metric_value = rank
    instances_tested = 1
    conjecture_holds = 0.7 <= ratio <= 1.3
    counterexample = "" if conjecture_holds else f"rank={rank}, expected={expected_rank}"
```

```

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        for result in results:
            if not result["conjecture_holds"]:
                counterexample = result["counterexample"]
                first_failing_seed = seed
                break
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

TRIAL: {'metric_name': 'categorified_symplectic_form_rank', 'metric_value': 662509, 'instances_tested': 1000000}
TRIAL: {'metric_name': 'categorified_symplectic_form_rank', 'metric_value': 20721, 'instances_tested': 1000000}

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

Safety rail: critic\_challenge\_falsified | original: The categorified symplectic form rank of the given Boolean formulas does not satisfy the expected ratio within the specified range for at least 80% of | next: Investigate the nature of these counterexamples to determine if there is a pattern or if they represent a general failure of the conjecture.

## 11. Audit log (LLM calls)

**Total LLM calls:** 11

| #  | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|----|-----------------|---------------|------------------|-----------|-----|--------------|
| 1  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 12072        |
| 2  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 11893        |
| 3  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 11749        |
| 4  | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 5785         |
| 5  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 4677         |
| 6  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 5351         |
| 7  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 13135        |
| 8  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10171        |
| 9  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10554        |
| 10 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 7521         |
| 11 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 57441        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 150350 ms total latency. Provider mix: {'ollama\_remote': 11}

(full prompt+response transcripts available in *research/audit/15f19a3d4b30.jsonl*)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/15f19a3d4b30.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** *research/pvsnp\_sandbox/test\_\*.py* (per-cycle)

**Replay tarball:** *research/replay/15f19a3d4b30.tar.gz* (if generated)